

Learned Page Replacement for xv6

Training on verified reference streams, and what a kernel can actually use

traces2 study — report/ML_REPORT.md

The question

Can a learned model replace the kernel's page-replacement policy?

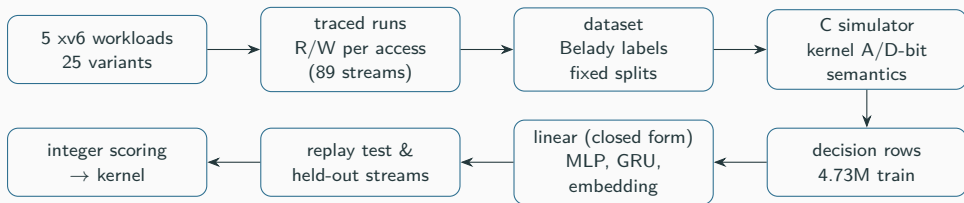
Train on memory workloads, integrate the model into xv6, measure it — a precursor to Linux.

Three constraints shape everything:

1. **No floating point in xv6.** Anything learned must run in integers.
2. **A kernel never sees memory accesses.** The MMU handles loads and stores; the kernel sees faults and the accessed/dirty bits it can read at a scan. Same in Linux for anonymous memory.
3. **Decisions compound.** Policies can only be compared by replaying whole reference streams — all results are fault counts, not prediction accuracy.

So every model is trained on three **feature tiers**: what only an oracle trace has, what a kernel observes, and what a kernel can cheaply book-keep — the first is an *ablation*, not a deployable result.

Pipeline



How the traces were made

The workloads

Workload	Models	Why no single heuristic wins
btreebench	B+tree (SQLite pager): insert, lookup, scan, mixed, WAL	lookups favour frequency, inserts recency, scans sequential
kvbench	Zipfian hash table (Redis): YCSB modes A/B/C/D/F, rehash, value sizes	hot set rotates: old-hot pages must go cold
graphbench	BFS + fixed-point PageRank on a scale-free graph	data-dependent frontier; hubs revisited each iteration
sortbench	bottom-up merge sort	deliberately sequential (calibration)
matmulbench	naive vs. blocked matrix multiply	locality contrast, tiny working sets

Collecting a complete, verified reference string

- Every arena access goes through `vmbench_trace_ref(vpn, access)`: one line R <vpn> or W <vpn> per access — every touch, not just faults. No default access type: an unmarked site does not compile.
- A reference string does not depend on memory size, so each stream is **one run at a generous margin**; capacities are chosen in simulation.
- 25 variants $\times$ seeds = **89 streams** + 22 repeats, 6 parallel lanes. A stream counts only if the run passed and the file matches `TRACEEND refs=N bytes=M` exactly.
- All 111 runs passed; all 22 repeats **byte-identical**.
- Checked against the kernel: FIFO replay reproduces its evictions with 3–5 untraced frames (−0.1%); W marks predict its `page_writes`.

Why the older traces were dropped

Two of the six old captures were wrong (found by AU5TRA, commit 9d36525):

- **graphbench** traced only the edge array — BFS `visited[]` and PageRank `rank[]` were never recorded: 57% of references, 667 of 2000 pages missing.
- **lzwbench**'s trace hit xv6's maximum file size and silently lost ~42% of the run while still printing PASS.

The earlier report's two headline wins (graph -4.3%, lzw -24% vs. Clock) were on these two traces. **They should not be cited.** Everything here uses the new streams.

The dataset and its splits

Per reference: `vpn`, `write`, `next_use`.

`next_use` = distance to the page's next access — what Belady evicts by. A **label only**, never an input.

held-out: variants never trained on

kv-F, btree-scan, graph-bfs2000, sort-n60000, matmul-128

test: last seed of every other variant

val: second-to-last seed

Split	Streams	References
train	41	45.5M
val	15	23.6M
test	15	23.6M
held-out	18	30.6M

Features, models, method

Three feature tiers

Tier	Feature	Definition	Linux analogue
F (oracle)	rec	references since last access	—
	freq	exact access count	—
	sd	distinct pages since last access	—
	wr	fraction of accesses that wrote	—
K (kernel)	ref	accessed bit at this scan	referenced / young bit
	aging	8-bit aging counter (xv6 Aging)	—
	sfreq	scans that found it accessed	DAMON <code>nr_accesses</code> , MGLRU tiers
	idle	scans since last found accessed	MGLRU generations, DAMON age
	age dirty	scans since loaded dirty bit	— PG_dirty
K+ (bookkeeping)	refaults	times evicted and faulted back	mm/workingset.c shadow entries
	rdist	evictions between eviction and refault	

Simulator, validated before use

`tools/ml2/pagesim.c`: replays a stream from empty memory; FIFO, Clock, Aging, LRU, exact/decayed LFU, Belady, and any learned scorer. Accessed/dirty bits follow the kernel (a fault sets PTE_A, and PTE_D on a write).

- FIFO, LRU, Belady equal AU5TRA's independent `baselines.tsv` on all 534 stream $\times$ capacity cells.
- Clock/Aging/decayed LFU match a naive Python re-implementation in faults and writebacks; all kernel features match on 11,792 evictions $\times$ 53 candidates.
- The C scorer reproduces every trained model (max error $\leq 3 \times 10^{-5}$).

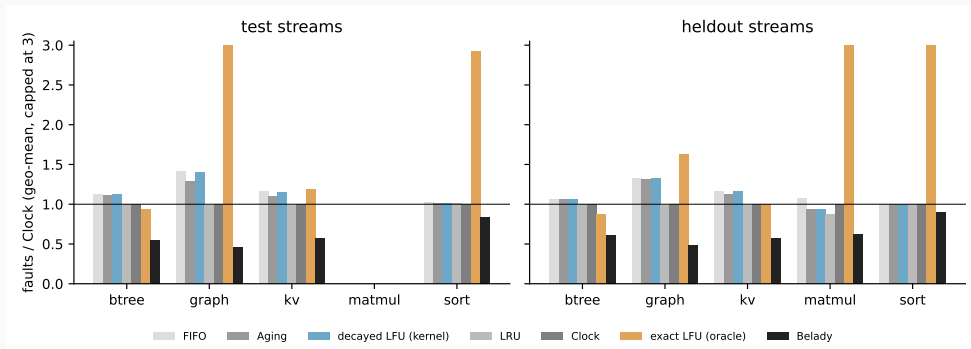
Two corrections to the earlier study: its “stack distance” counted first-ever touches, not distinct pages (true stack distance orders pages exactly like LRU); and pooled correlation misled — what matters is ranking *within* a decision.

Training on decisions

- Replay every train/val stream at 5, 10, 20% of its distinct pages under LRU; at sampled evictions record 32 resident candidates — Belady's choice, the 4 most recently used, and random others — with all 12 features and $\log(1 + \text{distance to next use})$.
- **4.73M rows over 357K eviction decisions**; 0.84M validation rows.
- Models: **linear** (closed form, any subset), **MLP** 2×32 , a **ranking loss** (softmax over one decision's candidates), **GRU** over the last 8 access intervals, **cross-page embedding**.
- Scopes: one model per workload, and one **global** model — the realistic one: a kernel does not know which program is running.
- **Every combination**: $300 \text{ feature sets} \times 6 \text{ scopes} = 28,800$ full replays at 10%.
- **Nested selection on validation**: top-5 subsets per tier $\times$ probation $\{0, 1, 2, 4\}$ at 5/10/20%; only the winner is run on test and held-out.

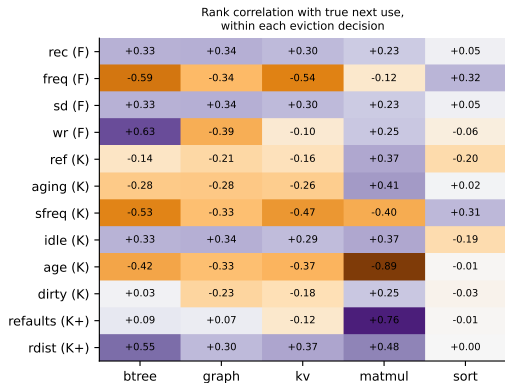
Results

What xv6 can already do: the classical policies



Faults relative to Clock (xv6's best), geo-mean over streams $\times 5/10/20\%$. Aging $\approx$ FIFO; exact LFU wins on btree and is catastrophic elsewhere; Belady needs 43–54% fewer faults on btree/kv/graph.

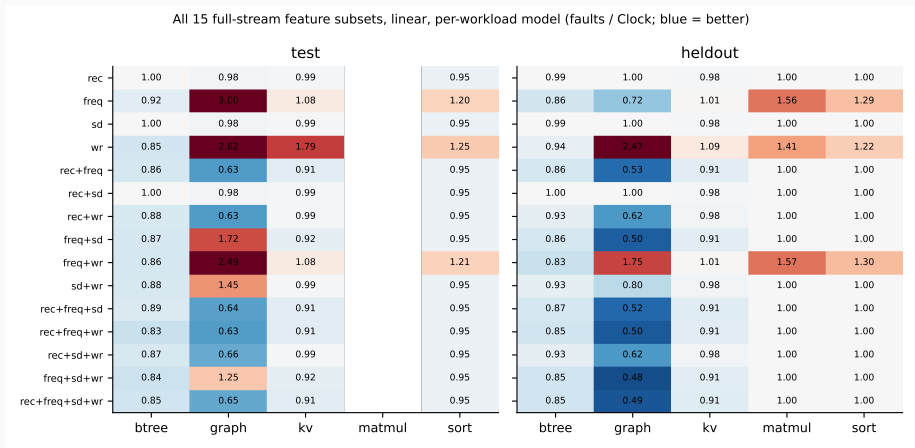
Which features carry signal — within one eviction



Rank correlation with the true next use, *among the candidates of one decision*.

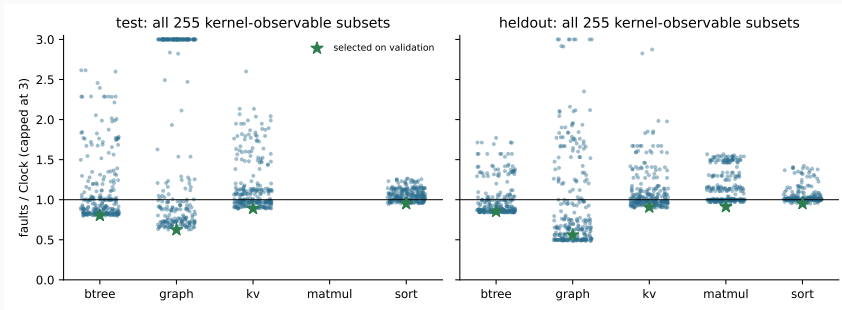
- Sampled access count `sfreq` (-0.45) $\approx$ exact count `freq` (-0.52).
- Refault distance `rdist` — Linux already measures it — is as strong.
- Recency and true stack distance are identical within a decision.
- Pooled correlation misleads: `rec` on `sort` is $+0.87$ pooled, $+0.05$ within decisions.

Every full-stream combination (15 subsets)



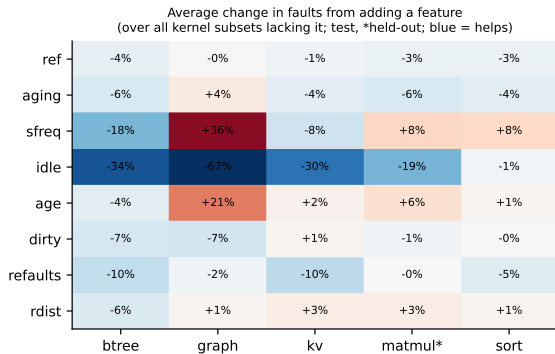
One feature is never enough; **recency + frequency** is the pair that matters.

All 255 kernel-observable combinations



workload	best	10th pct	median	90th pct	beat Clock	$\geq 3\times$ Clock
btree	0.797	0.820	0.932	1.772	152/255	0
graph	0.628	0.681	1.004	3.000	126/255	98
kv	0.888	0.907	1.012	1.759	122/255	0
sort	0.945	0.967	1.046	1.157	71/255	0

What each kernel feature is worth



Median change in faults when a feature is added to a subset that lacks it.

- `idle` — scans since last found accessed — dominates: **-71% graph**, **-29% btree**, **-20% kv**.
- That is exactly what Linux's MGLRU generations and DAMON age track.
- Refault count helps `btree/kv/sort`.

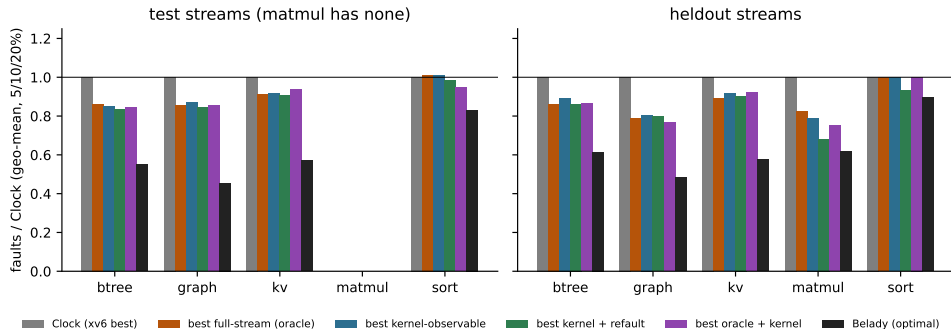
What went wrong first: evicting the page being streamed

v1 recorded Belady's choice + 31 *uniform* candidates per decision. On PageRank almost every model was $\geq 3\times$ **Clock** — even one trained on PageRank alone; three DAgger rounds did not help.

- At its own decisions: **97% of victims loaded at the previous fault**, next use a median **16 references** away (Belady's choice: ~ 300 refs stale).
- Uniform samples of a 167-page resident set almost never contain the page just touched — no model ever saw “just used, keep it”.
- Same pathology as the kernel LFU livelock, by a different route.

Fixes, both chosen on validation: record the 4 most recently used candidates too; **probation** — a page loaded $< p$ scans ago is not evicted while an older one exists (Clock's second chance). Graph: $0.81 \rightarrow 0.61$; matmul prefers $p = 0$ (5 frames), so p is chosen per scope.

Headline: validation-selected models, per workload



Subset and probation chosen on validation only; geo-mean over 5/10/20%.

Headline numbers

workload	LRU	oracle (F)	kernel (K)	kernel+refault (KUK+)	oracle+kernel	Belady
btree	0.994	0.861 (+32%)	0.852 (+33%)	0.833 (+38%)	0.845 (+35%)	0.550
graph	0.990	0.855 (+22%)	0.871 (+21%)	0.845 (+26%)	0.855 (+22%)	0.456
kv	0.986	0.910 (+21%)	0.917 (+19%)	0.905 (+22%)	0.937 (+15%)	0.571
sort	1.012	1.011 (+12%)	1.011 (+11%)	0.983 (+30%)	0.947 (+30%)	0.830

workload	LRU	oracle (F)	kernel (K)	kernel+refault (KUK+)	oracle+kernel	Belady
btree	0.997	0.861 (+38%)	0.893 (+29%)	0.863 (+37%)	0.864 (+37%)	0.615
graph	0.992	0.786 (+31%)	0.805 (+29%)	0.798 (+30%)	0.767 (+34%)	0.482
kv	0.982	0.890 (+27%)	0.916 (+21%)	0.905 (+23%)	0.922 (+19%)	0.577
matmul	0.879	0.823 (+26%)	0.789 (+39%)	0.681 (+73%)	0.751 (+44%)	0.617
sort	0.999	0.999 (+4%)	0.999 (+4%)	0.933 (+51%)	1.001 (+0%)	0.896

faults / Clock (share of the Clock→Belady gap closed)

The kernel's own signals are enough

- With only accessed/dirty-bit scan history + refault bookkeeping, the selected policy **beats Clock on every workload**, on unseen seeds *and* unseen variants:
btree -17/ - 14%, graph -15/ - 20%, kv -10/ - 10%, matmul -32%, sort -2/ - 7% (test / held-out).
- The oracle features are within $\pm 1.5\%$ of it on btree/graph/kv and behind on matmul and sort:
what a kernel cannot see is not what limits a learned policy here.
- Refault bookkeeping helps in all 9 workload $\times$ split cells.
- Held-out variants generalise as well as unseen seeds.
- **One global model** (the deployable case) still beats Clock in 8 of 9 cells, keeping most of the gain on btree/graph/kv.

Fewer disk writes, not just fewer faults

workload	split	LRU	kernel+refault	oracle	Belady
btree	test	0.997	0.967	0.932	0.707
btree	heldout	0.997	0.965	0.962	0.646
graph	test	0.938	0.418	0.578	0.257
graph	heldout	0.727	0.338	0.616	0.297
kv	test	0.975	0.819	0.856	0.690
kv	heldout	0.982	0.904	0.890	0.574
matmul	heldout	1.008	0.658	1.008	0.863
sort	test	1.016	0.987	1.015	0.868
sort	heldout	1.000	0.935	1.000	0.932

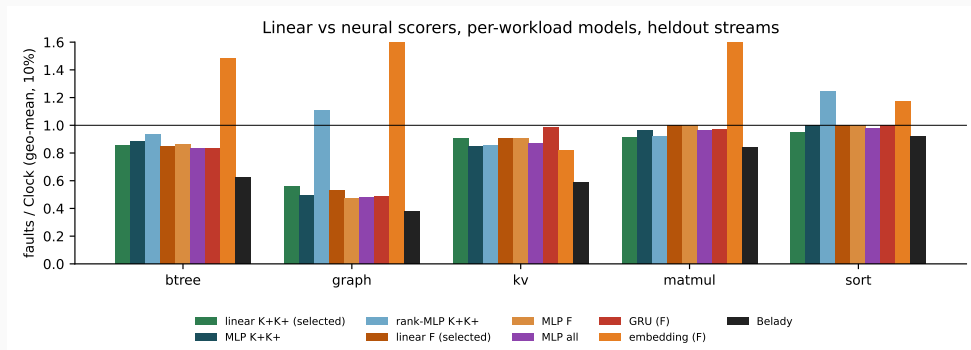
The selected models use the dirty bit: graph needs 58–66% fewer page writes than Clock (Belady, which ignores writes: 70–74%).

Where it does not help: capacity dependence

workload	split	5%	10%	20%	Belady 5%	Belady 10%	Belady 20%
btree	test	0.804	0.803	0.895	0.608	0.550	0.498
btree	heldout	0.853	0.854	0.882	0.686	0.627	0.541
graph	test	0.979	0.624	0.986	0.412	0.337	0.682
graph	heldout	0.931	0.560	0.976	0.477	0.377	0.623
kv	test	0.867	0.890	0.960	0.629	0.573	0.516
kv	heldout	0.841	0.907	0.970	0.617	0.588	0.530
matmul	heldout	0.500	0.913	0.692	0.500	0.839	0.560
sort	test	1.099	0.950	0.912	0.758	0.897	0.842
sort	heldout	0.994	0.951	0.859	0.959	0.918	0.818

Graph's gain is concentrated at 10%; at 5% and 20% no linear model gets below 0.93 even though Belady has large headroom — including models trained only on that capacity.

Neural, sequence and ranking models



A linear score is competitive with everything: MLP gains on kv and held-out graph only; the oracle GRU adds nothing; the cross-page embedding is catastrophic on btree/graph but best on held-out kv; the ranking loss did not help.

Can it run without floating point?

workload	split	float	int, 4-bit	int, 8-bit	int, 12-bit
btree	test	0.833	0.834	0.833	0.833
btree	heldout	0.863	0.875	0.862	0.863
graph	test	0.845	0.848	0.845	0.845
graph	heldout	0.798	0.812	0.799	0.798
kv	test	0.905	0.905	0.905	0.904
kv	heldout	0.905	0.903	0.904	0.904
matmul	heldout	0.681	0.681	0.681	0.681
sort	test	0.983	0.983	0.983	0.983
sort	heldout	0.933	0.932	0.933	0.933

With 8-bit weights and Q8 features every selected model is within ± 0.002 of float (most identical); 4-bit stays within $\sim 2\%$. The policy runs in xv6 as trained — no floating point.

Does the training distribution matter? (one DAgger round)

workload	split	trained under LRU	after DAgger
btree	test	0.833	0.844
btree	heldout	0.863	0.898
graph	test	0.845	0.889
graph	heldout	0.798	0.804
kv	test	0.905	0.905
kv	heldout	0.905	0.905
matmul	heldout	0.681	0.701
sort	test	0.983	0.998
sort	heldout	0.933	0.951

Re-recording under the model's own decisions and refitting does **not** help: most selected models are unchanged or slightly worse. With stratified recording and probation, LRU-recorded decisions already cover the states these policies meet.

In xv6 itself

VM_POLICY_ML: the learned policy in the kernel

- `choose_ml()` in `kernel/vmpage.c`: scan candidates (read/clear PTE_A), integer score $\sum_j q_j X_j$, evict the highest; probation for new pages.
- Features from `kernel/mlfeat.h`, **shared with the simulator**: table `log1p`, Q8, no floating point, no `libgcc`.
- Refault bookkeeping per swap slot (Linux's shadow entry): eviction scan + refault count.
- Weights **per process** (`vmctl(VM_SET_ML_WEIGHTS)`, inherited on fork/exec); `vmrun ml kv kvbench...`
- Passes data-invariance with all five policies (byte-identical data).
- Fixed on the way: LFU livelock (decayed counter); benchmark burn phase gave each policy a different memory size (now always FIFO).

231 runs: 33 test/held-out streams $\times$ 7 policies, 10% memory, kernel counters.

In xv6: faults relative to Clock

Test (unseen seeds)

workload	FIFO	Aging	LFU (decayed)	ML global	ML per-workload	ML per-workload, K only
btree	1.143	1.116	1.127	0.843	0.823	0.824
graph	2.096	1.585	2.027	0.989	0.614	0.630
kv	1.284	1.107	1.175	0.875	0.871	0.908
sort	1.331	0.988	0.991	0.980	0.960	0.988

Held-out (unseen variants)

workload	FIFO	Aging	LFU (decayed)	ML global	ML per-workload	ML per-workload, K only
btree	1.087	1.063	1.073	0.893	0.854	0.893
graph	1.289	1.264	1.274	0.535	0.541	0.550
kv	1.312	1.133	1.187	0.878	0.875	0.924
matmul	1.820	0.999	0.999	1.115	0.973	0.999
sort	1.190	0.998	1.006	0.977	0.933	0.998

Per-workload weights **beat Clock on every workload**: btree $-18/-15\%$, graph $-39/-46\%$, kv -13% , sort $-4/-7\%$, matmul -3% . The global model **loses on matmul** ($+12\%$).

In xv6: page writes relative to Clock

Test						
workload	FIFO	Aging	LFU (decayed)	ML global	ML per-workload	ML per-workload, K only
btree	1.058	1.026	1.028	0.952	1.022	1.043
graph	2.533	1.750	2.301	0.515	0.045	0.045
kv	1.992	1.462	1.645	0.737	0.682	0.715
sort	1.148	0.982	0.987	0.965	0.944	0.982

Held-out						
workload	FIFO	Aging	LFU (decayed)	ML global	ML per-workload	ML per-workload, K only
btree	1.069	1.018	1.019	0.984	0.966	0.981
graph	1.375	1.323	1.332	0.395	0.138	0.130
kv	1.221	1.133	1.171	0.878	0.875	0.924
matmul	13.002	0.987	0.987	3.889	0.035	0.987
sort	1.097	0.995	1.010	0.971	0.925	0.995

Graph 95% / 86% fewer writes, matmul 96%, kv 32% (test) — the dirty bit at work.

Did the simulator predict the kernel?

workload	split	FIFO	ML global	ML per-workload
btree	test	1.143 → 1.121	0.843 → 0.841	0.823 → 0.804
btree	heldout	1.087 → 1.068	0.893 → 0.893	0.854 → 0.853
graph	test	2.096 → 1.632	0.989 → 0.659	0.614 → 0.625
graph	heldout	1.289 → 1.262	0.535 → 0.515	0.541 → 0.561
kv	test	1.284 → 1.159	0.875 → 0.891	0.871 → 0.889
kv	heldout	1.312 → 1.174	0.878 → 0.901	0.875 → 0.905
matmul	heldout	1.820 → 1.338	1.115 → 1.338	0.973 → 0.913
sort	test	1.331 → 0.976	0.980 → 0.967	0.960 → 0.950
sort	heldout	1.190 → 1.000	0.977 → 0.986	0.933 → 0.951

Kernel → simulator, faults/Clock for the same runs: **agreement to a few hundredths** for the ML models. FIFO is worse in the kernel (it also evicts hot code/stack pages the traces lack). Graph test differs because of one stream ...

...that sits on a capacity cliff

policy	kernel	sim +0	sim +1	sim +2	sim +3	sim +4
Clock	23390	54948	39982	23609	12041	6758
FIFO	86130	100806	92613	85126	78665	72553
Aging	41625	91873	73061	42768	13805	4742
LFU (decayed)	84009	100942	92095	84006	76674	69405
ML global	65576	53145	36474	20121	9525	6577
ML per-workload	18309	48118	32800	16382	7436	5327
ML per-workload, K only	19842	54067	36946	17081	7351	5245

The kernel gives this PageRank stream ≈ 2 extra frames; at +2 the simulator matches every classical policy within 3%. Per-workload ML still beats Clock there; the global model **does not** (cause not established).

The price: victim-selection time

policy	ticks / eviction	candidates / eviction
Clock	35.3	106.0
FIFO	12.6	106.0
Aging	199.6	106.0
LFU (decayed)	199.4	106.0
ML global	411.0	106.0
ML per-workload	451.7	106.0
ML per-workload, K only	400.6	106.0

ML visits every resident page on each eviction: $11\text{--}13\times$ Clock, $\approx 2\times$ Aging ($\approx 40\mu\text{s}$ emulated). Small next to a disk read in xv6; a real kernel would score a sample or batch (as MGLRU does).

Conclusions

Feature that won here	Linux already has
idle (scans since last found accessed)	MGLRU generations; DAMON region age
sfreq (scans found accessed)	DAMON nr_accesses; MGLRU tiers
dirty	PG_dirty
refaults, rdist	workingset shadow entries (mm/workingset.c)
rec, freq, sd (oracle)	nothing without per-access instrumentation

The winning signals are ones Linux maintains; the score is a few integer multiply-adds per candidate in a scan the kernel already does.

Takeaways

1. On verified reference streams, a **linear score over 4–5 kernel-observable features** beats xv6's best policy on all five workloads, on unseen seeds and unseen variants — 2–32% fewer faults (10–32% on btree, graph, kv, matmul), up to 66% fewer disk writes.
2. **Oracle information buys almost nothing** over those features; `idle` (the kernel's own recency) carries most of the value.
3. **Bigger models do not pay**: MLP, GRU, cross-page embedding, ranking loss are no better than linear except MLPs on kv and held-out graph.
4. **Failure modes are real**: many feature sets are catastrophic, and the first training set taught models to evict the page being streamed. Validation-based selection and probation are part of the method, not details.
5. **It works in the kernel**: integer-only `VM_POLICY_ML` beats Clock in xv6 on all five workloads (3–46% fewer faults, up to 96% fewer writes), as the simulator predicted; selection costs 11–13× Clock.
6. The earlier graph and lzw results were measured on broken traces and are withdrawn.

Next steps

1. Cheaper selection: score a sample/batch of candidates instead of all of them.
2. In-kernel runs at 5% and 20% memory (mechanical; hours of QEMU time).
3. Pick weights automatically per program or phase (the global model loses on matmul).
4. Capacity-aware models for graph at 5% and 20%.
5. Port to Linux: the same features from MGLRU / DAMON / workingset (plan in the report, §9.1).